Chapter 12

sqlite3: a database in a file

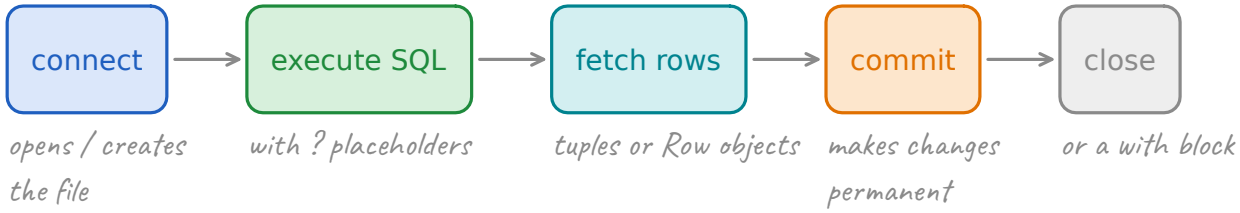
a file you can ask questions, that survives crashes

Sooner or later a program needs to remember things between runs: users, bookings, scores, loans. A JSON file works until you need to change one record without rewriting the lot, find "every loan overdue by a week", or survive the power going off halfway through a save.

SQLite is a full SQL database that lives in **one ordinary file** and ships inside Python as `sqlite3`. No server, no install. It's in every phone and browser, and it's the right first database for almost any project.

need	JSON file	SQLite
change one record	rewrite the whole file	`UPDATE` touches one row
"which loans are overdue?"	load everything, loop in Python	one `SELECT ... WHERE`
crash halfway through a save	file can be left half-written	transaction: all or nothing
two programs at once	last writer wins, silently	locking handles it
rules ("email is unique")	your code must remember to check	the database refuses bad rows

The lifecycle you'll use every time:



Your first database, end to end

```python
import sqlite3

conn = sqlite3.connect("library.db")
conn.execute("""CREATE TABLE IF NOT EXISTS books (
                    id     INTEGER PRIMARY KEY,
                    title  TEXT NOT NULL,
                    year   INTEGER)""")
conn.execute("INSERT INTO books (title, year) VALUES (?, ?)",
             ("Dune", 1965))
conn.commit()
conn.close()

conn = sqlite3.connect("library.db")
print(conn.execute("SELECT * FROM books").fetchall())
conn.close()
```

← 1. connect (creates the file)

← 2. execute SQL

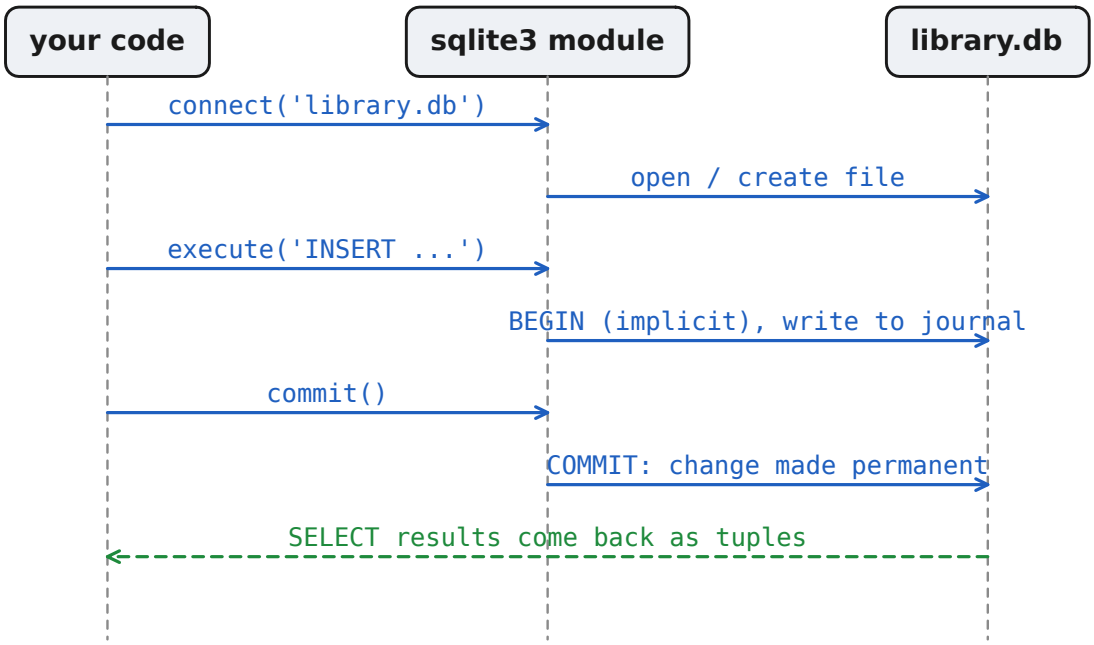
← 3. commit: now on disk

4. close

a later run: still there

```
[(1, 'Dune', 1965)]
```

That's the whole library. Everything else in this chapter is SQL (the language you send through `execute`) and the habits that keep it safe. Here's what happens between your program and the file:

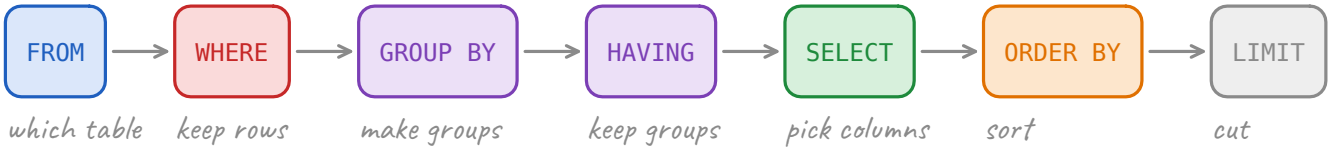


TRAP

Close without `commit()` and your inserts are **silently thrown away**. sqlite3 opens a transaction automatically before the first `INSERT`/`UPDATE`/`DELETE`, and only `commit()` ends it. (The `with conn:` form in a few pages does this for you.)

SQL in one picture: a query is a pipeline

You write a query in one order, but the database **runs** it in another. Read it in run order and SQL stops being mysterious:



Eight books are already in a table. Every grid below is the real result:

```sql
SELECT title, author, year FROM books WHERE year > 1900 ORDER BY year LIMIT 4
```

title	author	year
Dune	Herbert	1965
Children of Dune	Herbert	1976
Neuromancer	Gibson	1984
Count Zero	Gibson	1986

```sql
SELECT author, COUNT(*) AS n_books, SUM(copies) AS copies
FROM books
GROUP BY author
HAVING COUNT(*) > 1
ORDER BY n_books DESC, author
```

author	n_books	copies
Austen	3	6
Gibson	2	1
Herbert	2	3

✎ YOUR GO (pencil, or the REPL, first)

Before reading on, predict the rows: SELECT title FROM books WHERE copies = 0 OR author = 'Simmons'. Then: why can't WHERE use the alias n_books? (Look at the pipeline.)

```sql
SELECT title FROM books WHERE copies = 0 OR author = 'Simmons'
```

title
Count Zero
Hyperion

clause	job	tiny example
SELECT	columns or expressions to return	SELECT title, year + 1 AS next
WHERE	filter rows (before grouping)	WHERE year < 1990
GROUP BY	one output row per group	GROUP BY author
HAVING	filter groups (after grouping)	HAVING COUNT(*) > 1

`ORDER BY`	sort; `DESC` reverses	`ORDER BY year DESC, title`
`LIMIT`	first n rows (`OFFSET` skips)	`LIMIT 10 OFFSET 20` (page 3)

Changing data: INSERT, UPDATE, DELETE

```sql
UPDATE books SET copies = copies + 1 WHERE title = 'Count Zero'
```

```
ok: 1 row changed
```

```sql
DELETE FROM books WHERE year < 1812
```

```
ok: 1 row changed
```

```sql
SELECT title, copies FROM books WHERE author = 'Austen' OR title = 'Count Zero'
```

title	copies
Emma	3
Persuasion	1
Count Zero	1

TRAP

An `UPDATE` or `DELETE` with no `WHERE` changes **every row**. Write the `WHERE` first, and try it as a `SELECT` before turning it into an `UPDATE`.

Placeholders, and the attack they stop

The single most important habit in this chapter. Never build SQL by pasting values into the string. Here's a login check written the wrong way, and what an attacker types as the username:

```
import sqlite3
users = sqlite3.connect(":memory:")
users.execute("CREATE TABLE users (name TEXT, pw TEXT)")
users.executemany("INSERT INTO users VALUES (?, ?)",
                  [("alice", "x1"), ("bob", "x2")])

def find_user_BAD(name):
    q = f"SELECT name FROM users WHERE name = '{name}'"
    print("   runs:", q)
    return users.execute(q).fetchall()

print(find_user_BAD("alice"))
print(find_user_BAD("nobody' OR '1'='1"))
```

← value pasted into SQL

← attacker's 'name'

```
   runs: SELECT name FROM users WHERE name = 'alice'
[('alice',)]
   runs: SELECT name FROM users WHERE name = 'nobody' OR '1'…
[('alice',), ('bob',)]
```

The quote in the input **closed the string early**, and the rest became SQL: `OR '1'='1'` is true for every row, so the "login" matched everyone. With a placeholder, the value travels separately from the SQL and can never become code:

```
def find_user(name):
    q = "SELECT name FROM users WHERE name = ?"
    return users.execute(q, (name,)).fetchall()

print(find_user("alice"))
print(find_user("nobody' OR '1'='1"))
print(find_user("O'Brien"))
```

← (name,) is a 1-tuple

← quotes just work, too

```
[('alice',)]
[]
[]
```

placeholder style	looks like	params
qmark	`name = ? AND year = ?`	a tuple, in order: `(name, year)`
named	`name = :n AND year = :y`	a dict: `{"n": name, "y": 1965}`

TRAP

Placeholders are for **values only**. A table or column name can't be a `?`. If one has to vary, check it against a fixed list (`if col not in {"title", "year"}: raise ValueError`) before putting it in the string. And `(name)` is not a tuple; `(name,)` is.

Getting rows out

`execute` returns a **cursor**: a pointer into the result that you read from. Four ways to read it:

```
cur = conn.execute("SELECT title, year FROM books ORDER BY year")
print(cur.fetchone())            # the next row, or None at the end
print(cur.fetchmany(2))          # the next 2 rows
print(len(cur.fetchall()))       # everything left

for title, year in conn.execute("SELECT title, year FROM books LIMIT 2"):
    print(title, year)           # iterate: one row at a time, low memory
```

```
('Emma', 1815)
[('Persuasion', 1817), ('Dune', 1965)]
4
Dune 1965
Neuromancer 1984
```

Tuples are fine for two columns, but `row[3]` is unreadable. Set `row_factory = sqlite3.Row` and rows work by name as well as position:

```
import sqlite3
conn.row_factory = sqlite3.Row
row = conn.execute("SELECT * FROM books WHERE title = ?", ("Dune",)).fetchone()
print(row["title"], row["year"], row.keys())
print(dict(row))
conn.row_factory = None
```

```
Dune 1965 ['id', 'title', 'author', 'year', 'copies']
{'id': 1, 'title': 'Dune', 'author': 'Herbert', 'year': 1965, 'copies': 2}
```

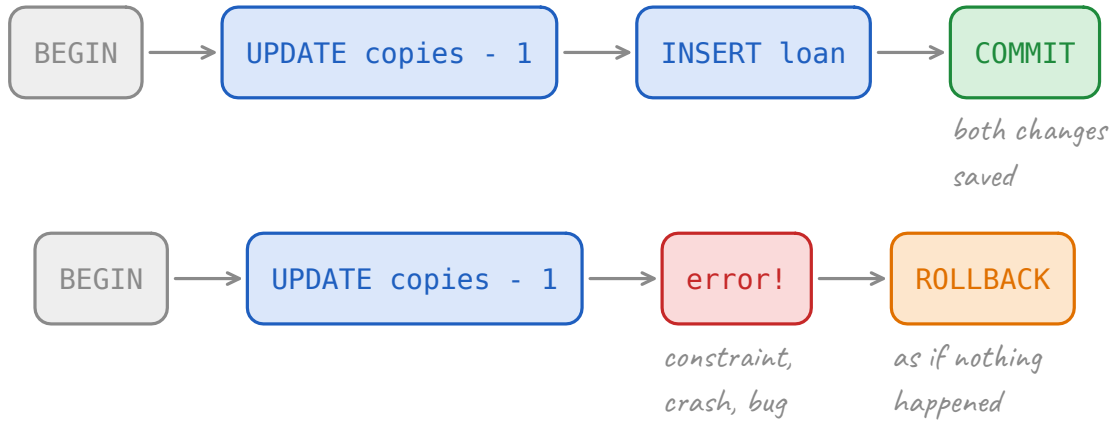
Insert many rows at once with `executemany`, and get the id SQLite gave a new row with `lastrowid`:

```
new = [("Ubik", "Dick", 1969, 1), ("Solaris", "Lem", 1961, 2)]
conn.executemany("INSERT INTO books (title, author, year, copies) "
                 "VALUES (?, ?, ?, ?)", new)
cur = conn.execute("INSERT INTO books (title, author) VALUES (?, ?)",
                   ("Kindred", "Butler"))
print("new id:", cur.lastrowid)
conn.commit()
```

```
new id: 10
```

Transactions: all or nothing

Lending a book is two changes: take a copy off the shelf **and** record the loan. If the program crashes between them, the copy has vanished and nobody has it. A **transaction** groups changes so they all happen or none do.



`with conn:` is the tidy way: it commits if the block finishes and rolls back if an exception escapes. Here the `CHECK (copies >= 0)` rule fires on the second loan of a one-copy book:

```
conn.execute("CREATE TABLE IF NOT EXISTS loans (book_id INTEGER, who TEXT)")
conn.commit()

def lend(book_id, who):
    with conn:                                  # one transaction
        conn.execute("INSERT INTO loans VALUES (?, ?)", (book_id, who))
        conn.execute("UPDATE books SET copies = copies - 1 WHERE id = ?", (book_id,))

nid = conn.execute("SELECT id FROM books WHERE title='Neuromancer'").fetchone()[0]
lend(nid, "ana")
try:
    lend(nid, "ben")                            # no copies left
except sqlite3.IntegrityError as e:
    print("refused:", e)
print(conn.execute("SELECT who FROM loans").fetchall())
print(conn.execute("SELECT copies FROM books WHERE id=?", (nid,)).fetchone())
```

```
refused: CHECK constraint failed: copies >= 0
[('ana',)]
(0,)
```

Ben's loan row was inserted first, then the `UPDATE` failed, and the rollback removed his loan too. The database never shows half a loan.

TRAP

`with conn:` commits or rolls back, but it does **not close** the connection (unlike `with open(...)`). Close it yourself, or use `contextlib.closing(sqlite3.connect(...))`.

HONEST NOTE

Python 3.12 added `sqlite3.connect(..., autocommit=False)`, which gives PEP 249 behaviour (always in a transaction; you commit). The default you've seen here is the older "legacy" mode, and it's what most existing code and tutorials assume.

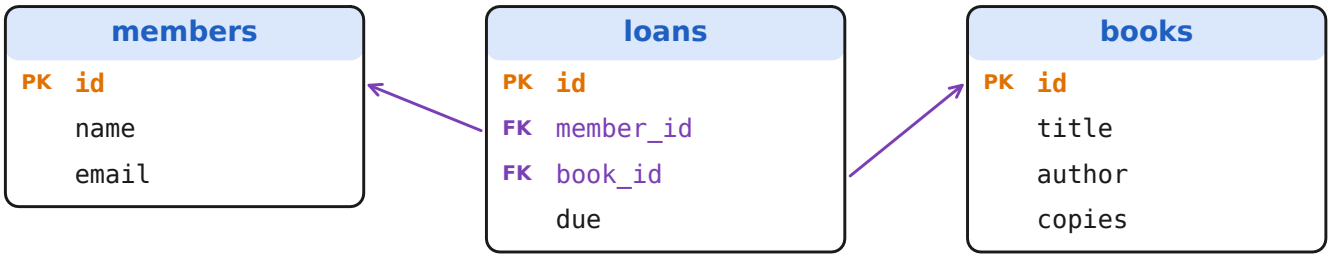
Designing tables: one fact in one place

The first design everyone writes is one wide table. It works until a fact needs to change:

```sql
SELECT * FROM loans_flat
```

member	email	book	due
Ana Silva	ana@x.org	Dune	2026-10-01
Ana Silva	ana@x.org	Emma	2026-10-03
Ben Okafor	ben@x.org	Dune	2026-10-02
Ana Silva	ana@old.org	Ubik	2026-10-09

Ana's email is stored three times, and one copy is already out of date. Which is right? This is an **update anomaly**. The fix is **normalisation**: each fact is stored once, in the table it belongs to, and other tables **refer to it by id**.



constraint	the database refuses...	error
`PRIMARY KEY`	a duplicate or missing id (`INTEGER PRIMARY KEY` auto-numbers)	IntegrityError
`NOT NULL`	a missing value	IntegrityError
`UNIQUE`	a second row with the same value (two members, one email)	IntegrityError
`CHECK (expr)`	a row where expr is false (`copies >= 0`)	IntegrityError
`REFERENCES t(id)`	an id that doesn't exist in `t`, **only if** `PRAGMA foreign_keys = ON`	IntegrityError

```sql
INSERT INTO loans2 (member_id, book_id, due) VALUES (99, 1, '2026-11-01')
```

```
sqlite3.IntegrityError: FOREIGN KEY constraint failed
```

TRAP
SQLite ignores `REFERENCES` unless you run `PRAGMA foreign_keys = ON` on **every new connection**, straight after connecting (it does nothing inside an open transaction). Put it in your `connect()` helper and forget about it.

JOIN: putting the facts back together

Normalised tables are split apart; a **join** glues them back for one query. `JOIN ... ON` pairs every row of one table with the rows of the other where the condition holds:

```sql
SELECT m.name, b.title, l.due
FROM loans2 AS l
JOIN members AS m ON m.id = l.member_id
JOIN books   AS b ON b.id = l.book_id
ORDER BY l.due
```

name	title	due
Ana Silva	Dune	2026-10-01
Ben Okafor	Dune	2026-10-02
Ana Silva	Emma	2026-10-03

Cat has no loans, so she isn't in that result: a plain (inner) join keeps only rows that match. To keep everyone from the left table, use `LEFT JOIN`; the missing side comes back as `NULL`, which `COUNT(column)` doesn't count:

```sql
SELECT m.name, COUNT(l.id) AS loans
FROM members AS m
LEFT JOIN loans2 AS l ON l.member_id = m.id
GROUP BY m.id
ORDER BY loans DESC
```

name	loans
Ana Silva	2
Ben Okafor	1
Cat Moreau	0

Let SQL do it

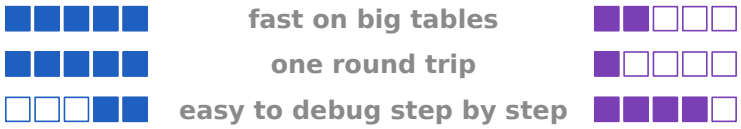
```
rows = conn.execute("""
    SELECT m.name, COUNT(l.id)
    FROM members m
    LEFT JOIN loans2 l
      ON l.member_id = m.id
    GROUP BY m.id
    ORDER BY m.name""").fetchall()
print(rows)
```

```
[('Ana Silva', 2), ('Ben Okafor', 1), ('Cat…
```

Fetch everything, loop in Python

```
counts = {}
for (mid, name) in conn.execute(
        "SELECT id, name FROM members"):
    counts[mid] = [name, 0]
for (mid,) in conn.execute(
        "SELECT member_id FROM loans2"):
    counts[mid][1] += 1
print(sorted(map(tuple, counts.values())))
```

```
[('Ana Silva', 2), ('Ben Okafor', 1), ('Cat…
```



RULE OF THUMB

Filter, join, count and sort **in SQL**, then fetch only what you need. Moving 100,000 rows into Python to count them is the classic beginner slowdown.

Indexes: why some queries are instant

To find rows matching a `WHERE`, SQLite either reads **every row** (a scan, O(n)) or jumps straight to them through an **index**, a sorted B-tree kept beside the table (O(log n)). `EXPLAIN QUERY PLAN` tells you which it chose:

```python
import sqlite3, time
big = sqlite3.connect(":memory:")
big.execute("CREATE TABLE visits (id INTEGER PRIMARY KEY, card TEXT, day TEXT)")
rows = ((f"C{i % 20000:05d}", f"2026-{i % 12 + 1:02d}-01")
        for i in range(200_000))
big.executemany("INSERT INTO visits (card, day) VALUES (?, ?)", rows)

def plan(q):
    return big.execute("EXPLAIN QUERY PLAN " + q, ("?",)).fetchone()[-1]

def timed(q, runs=200):
    t = time.perf_counter()
    for _ in range(runs):
        big.execute(q, ("C01234",)).fetchall()
    return (time.perf_counter() - t) / runs * 1000

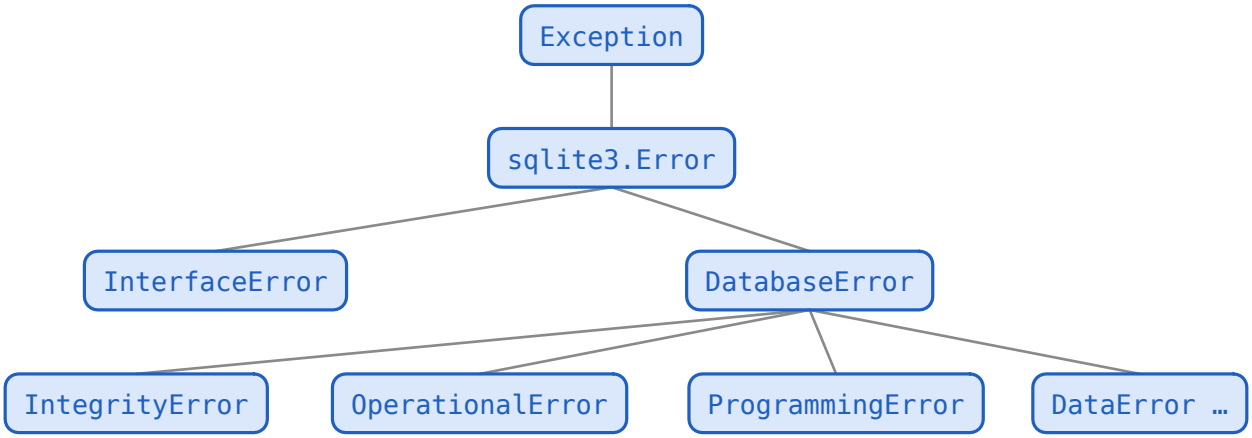
Q = "SELECT day FROM visits WHERE card = ?"
print(plan(Q), f"{timed(Q, 20):.2f} ms")
big.execute("CREATE INDEX idx_visits_card ON visits(card)")
print(plan(Q), f"{timed(Q):.3f} ms")
```

```
SCAN visits 5.52 ms
SEARCH visits USING INDEX idx_visits_card (card=?) 0.005 ms
```

Same query, same answer; the index turned a 200,000-row scan into a lookup that's hundreds to thousands of times faster. Primary keys and `UNIQUE` columns get an index automatically.

index a column when...	don't bother when...
you search or join on it often (`WHERE card = ?`, foreign keys)	the table is small (hundreds of rows)
you sort by it a lot (`ORDER BY due`)	the column has a handful of values (yes/no)
the table will grow large	the table is written far more than read (each index slows writes)

When it goes wrong: the error family



error	usually means	fix
IntegrityError	a constraint said no (UNIQUE, NOT NULL, CHECK, foreign key)	expected: catch it and tell the user
OperationalError	bad SQL, missing table, "database is locked", can't open file	a bug or an environment problem
ProgrammingError	wrong number of ? values, using a closed connection	a bug in your code

```sql
INSERT INTO members (name, email) VALUES ('Ana Two', 'ana@x.org')
```

```
sqlite3.IntegrityError: UNIQUE constraint failed: members.email
```

```sql
SELEC title FROM books
```

```
sqlite3.OperationalError: near "SELEC": syntax error
```

```sql
SELECT * FROM books WHERE id = ? AND year = ?
```

```
sqlite3.ProgrammingError: Incorrect number of bindings supplied. The current statemen…
```

RULE OF THUMB

Catch `IntegrityError` where you can say something useful ("that email is already registered"). Let `OperationalError` and `ProgrammingError` crash loudly during development: they're bugs, and hiding them makes them worse.

Organising database code

Scattering `conn.execute` through a program makes it hard to change. Keep all the SQL in one module with small functions that take a connection, and the rest of the program never sees SQL:

```python
import sqlite3
from dataclasses import dataclass

SCHEMA = """
CREATE TABLE IF NOT EXISTS members (
    id INTEGER PRIMARY KEY, name TEXT NOT NULL, email TEXT UNIQUE NOT NULL);
"""

@dataclass
class Member:
    id: int
    name: str
    email: str

def connect(path=":memory:"):
    conn = sqlite3.connect(path)
    conn.execute("PRAGMA foreign_keys = ON")
    conn.executescript(SCHEMA)
    return conn

def add_member(conn, name, email) -> int:
    with conn:
        return conn.execute("INSERT INTO members (name, email) VALUES (?, ?)",
                            (name, email)).lastrowid

def get_member(conn, member_id) -> Member | None:
    row = conn.execute("SELECT id, name, email FROM members WHERE id = ?",
                       (member_id,)).fetchone()
    return Member(*row) if row else None

conn = connect()
mid = add_member(conn, "Ana", "ana@x.org")
print(get_member(conn, mid), get_member(conn, 99))
```

```
Member(id=1, name='Ana', email='ana@x.org') None
```

habit	why
one `connect()` helper	every connection gets the same PRAGMAs and schema
functions take `conn` as a parameter	tests pass `":memory:"`; no global state
return dataclasses, not raw tuples	`m.email` beats `row[2]`; the rest of the code doesn't know it's SQL
schema version in `PRAGMA user_version`	a tiny migration system: `if version < 2: ALTER TABLE ...`
dates as ISO text `'2026-09-28'`	they sort correctly as strings; the old automatic date converters are deprecated since 3.12

The card, practice and stamp

LIBRARY CARD `sqlite3`

Job	A full SQL database in one file (or `":memory:"`), built into Python.
Core objects	`Connection` (from `sqlite3.connect`): `execute`, `executemany`, `commit`, `rollback`, `close`. `Cursor`: `fetchone`, `fetchall`, iterate, `lastrowid`. `sqlite3.Row` for rows by name.
Lifecycle	connect → `PRAGMA foreign_keys = ON` → execute with `?` → fetch → commit (or `with conn:`) → close.
Errors	`IntegrityError` (constraints), `OperationalError` (bad SQL, locked), `ProgrammingError` (wrong params, closed). All subclass `sqlite3.Error`.
Traps	**f-strings in SQL** (injection) · forgetting `commit` · `with conn` doesn't close · foreign keys off by default · `(x)` isn't a tuple.
Docs	docs.python.org/3/library/sqlite3.html · sqlite.org/lang.html (the SQL itself)

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 12.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 12.1` shows other valid solutions and their trade-offs.

`12.1` build from a spec A schema that refuses bad data
`12.2` fix the bug Close the injection hole
`12.3` build from a spec Top borrowers (JOIN + GROUP BY)
`12.4` build from a spec Lend and return, atomically
`12.5` measure and explain Make the slow queries use an index
`12.6` mini-project Mini-project: a library loans module

PASSPORT STAMP ask the same question of every new language (Appendix B)

10. I/O

Persistent structured data: `sqlite3` built in (one file, SQL, transactions). Other databases (PostgreSQL, MySQL) use the same DB-API shape: connect, cursor, execute, commit.

8. Errors

Errors from a library form a class tree (`sqlite3.Error` → `IntegrityError`...); catch the most specific one you can handle.

IN OTHER LANGUAGES

Every language has this exact shape. Java: JDBC `Connection`, `PreparedStatement` (the `?` placeholders), `commit()`. Go: `database/sql` with `db.Query(q, args...)`. The SQL and the injection rule are identical everywhere.

DEEP DIVE (your vault)

Vault notes: 11.1 Boundaries turn assumptions into failure modes · 19.12 Injection as a general grammar failure · 11.4 Context managers and deterministic release · 11.6 Buffers, flushing, and durability · 16.19 Relational joins and cardinality.